

ASESII 24A02 Project-Based Quiz 1

Ridge, Lasso, and Regularization for Neural Features

Group 05: [Bùi Trọng Thành - 24125043, Phan Anh Khoa - 24125059, Văn Tuấn Khải - 24125032, Huỳnh]

2026-07-17

Contents

Authorship and contribution statement	2
Abstract	2
Reproducible setup	2
Shared helper functions	2
1 Problem 1. Prediction design and leakage control	5
1.1 1A. Target and predictors	5
1.2 1B. Split and train-only preprocessing	6
1.3 1C. Training-data exploration	7
1.4 2A. OLS baseline	10
1.5 2B. Ridge	10
1.6 2C. Lasso	10
1.7 2D. Training-based comparison and locked core model	10
1.8 2E. Perturbation or stability check	10
2 Problem 3. Mathematical mechanisms	10
2.1 3A. Ridge and conditioning	10
2.2 3B. Lasso optimality and soft thresholding	10
2.3 3C. Connect algebra to evidence	10
3 Problem 4. Elastic net and a neural-feature bridge	11
3.1 4A. Research question and source map	11
3.2 4B. Elastic net on original predictors	11
3.3 4C. Fixed ReLU features	11
3.4 4D. Locked declaration and final holdout evaluation	12
3.5 4E. Deep-learning connection and limitations	12
4 Problem 5. Report quality and reproducibility	12
4.1 5A. Reproduction record	12
4.2 5B. Recommendation and limitations	12
4.3 5C. AI verification record	12

Authorship and contribution statement

We confirm that every group member can explain the submitted analysis. AI use is reported in GroupXX_AI_Log.csv, and the group checked every AI-assisted item used in this report.

Student	ID	Main contributions	Verification performed
[Name]	[ID]	[Work]	[Check]

Abstract

Maximum 150 words: prediction problem, methods, main evidence, and recommendation.

Reproducible setup

```
required_packages <- c("tidyverse", "glmnet", "broom", "knitr")
missing_packages <- required_packages[
  !vapply(required_packages, requireNamespace, logical(1), quietly = TRUE)
]
if (length(missing_packages)) {
  stop("Install the required packages before rendering: ",
       paste(missing_packages, collapse = ", "))
}

library(tidyverse)
library(glmnet)
library(broom)
library(knitr)

group_number <- as.integer(params$group_number)
if (length(group_number) != 1L || is.na(group_number) || group_number < 1L) {
  stop("Set params$group_number to your assigned positive group number.")
}

seeds <- list(
  split = 240200L + group_number,
  cv = 240300L + group_number,
  features = 240400L + group_number
)
```

Shared helper functions

```
find_exam_data <- function(filename) {
  input <- knitr::current_input(dir = TRUE)
```

```

input_dir <- if (length(input) && nzchar(input)) dirname(input) else getwd()
candidates <- c(
  file.path(input_dir, "data", filename),
  file.path(input_dir, "..", "data", filename),
  file.path(getwd(), "data", filename)
)
existing <- candidates[file.exists(candidates)]
if (!length(existing)) stop("Cannot locate ", filename, " by a relative path.")
hits <- unique(normalizePath(existing, winslash = "/", mustWork = TRUE))
if (length(hits) > 1L && length(unique(unname(tools::md5sum(hits)))) > 1L) {
  stop("Multiple nonidentical copies of ", filename, " were found.")
}
hits[[1L]]
}

split_rows <- function(n, train_prop = 0.80, seed) {
  stopifnot(n >= 10L, train_prop > 0, train_prop < 1)
  set.seed(seed)
  train <- sort(sample.int(n, floor(train_prop * n), replace = FALSE))
  list(train = train, test = setdiff(seq_len(n), train))
}

fit_scaler <- function(x, tol = 1e-12) {
  x <- as.matrix(x)
  center <- colMeans(x)
  centered <- sweep(x, 2L, center, "-")
  scale <- sqrt(colMeans(centered^2))
  keep <- is.finite(scale) & scale > tol
  if (!any(keep)) stop("No nonconstant training predictor remains.")
  list(
    center = center[keep],
    scale = scale[keep],
    keep = keep,
    dropped = colnames(x)[!keep]
  )
}

apply_scaler <- function(x, prep) {
  x <- as.matrix(x)[, prep$keep, drop = FALSE]
  x <- sweep(x, 2L, prep$center, "-")
  sweep(x, 2L, prep$scale, "/")
}

make_foldid <- function(n, k = 5L, seed) {
  if (k < 3L || k > n) stop("Require 3 <= k <= number of training rows.")
  set.seed(seed)
  sample(rep(seq_len(k), length.out = n))
}

```

```

}

fit_cv_glmnet <- function(x, y, alpha, foldid) {
  glmnet::cv.glmnet(
    x = x,
    y = y,
    family = "gaussian",
    alpha = alpha,
    foldid = foldid,
    standardize = FALSE,
    intercept = TRUE,
    type.measure = "mse"
  )
}

score_regression <- function(y, prediction) {
  y <- as.numeric(y)
  prediction <- as.numeric(prediction)
  if (length(y) != length(prediction) || any(!is.finite(c(y, prediction)))) {
    stop("Metrics require equal-length finite vectors.")
  }
  c(RMSE = sqrt(mean((y - prediction)^2)),
    MAE = mean(abs(y - prediction)))
}

count_nonzero <- function(fit, s = "lambda.min", tol = 1e-8) {
  b <- as.matrix(stats::coef(fit, s = s))
  slopes <- b[rownames(b) != "(Intercept)", 1L]
  sum(abs(slopes) > tol)
}

safe_condition_numbers <- function(x, lambda, tol = 1e-10) {
  eigenvalues <- eigen(
    crossprod(x) / nrow(x),
    symmetric = TRUE,
    only.values = TRUE
  )$values
  largest <- max(eigenvalues)
  smallest <- max(min(eigenvalues), 0)
  cutoff <- tol * max(1, largest)
  c(
    gram = if (smallest <= cutoff) Inf else largest / smallest,
    ridge = (largest + lambda) / (smallest + lambda)
  )
}

```

1 Problem 1. Prediction design and leakage control

1.1 1A. Target and predictors

```
config <- if (group_number %% 2L == 1L) {  
  list(  
    file = "fat.csv",  
    response = "brozek",  
    excluded = c("brozek", "siri", "density", "free")  
  )  
} else {  
  list(  
    file = "prostate.csv",  
    response = "lpsa",  
    excluded = "lpsa"  
  )  
}  
  
data_raw <- read.csv(find_exam_data(config$file), check.names = FALSE)  
predictors <- setdiff(names(data_raw), config$excluded)  
analysis_data <- data_raw[, c(config$response, predictors), drop = FALSE]  
  
if (anyNA(analysis_data)) stop("Declare a training-only missing-value plan.")  
if (!all(vapply(analysis_data, is.numeric, logical(1)))) {  
  stop("The assigned response and predictors must be numeric.")  
}
```

Define the prediction task and justify every exclusion. Target (Response): The prediction target is brozek.

Candidate Predictors: The available predictors are the remaining non-excluded anatomical measurements (e.g., age, weight, height, adipos, neck, chest, abdom, hip, thigh, knee, ankle, biceps, forearm, and wrist).

Unit of Observation: A single male subject.

Study Population: The population of men evaluated in the body-fat study described by Johnson (1996).

Prediction Objective: To predict an individual's body fat percentage (brozek) relying solely on simple, non-invasive anatomical measurements.

Dataset Source: The original study described by Johnson (1996).

Required Exclusions & Leakage Justification: We are required to explicitly exclude siri, density, and free. These specific variables encode alternative body-fat calculations or are physical quantities derived using body-fat information.

Including any outcome-derived variable fundamentally invalidates the prediction task by introducing target leakage. If included, the regularized models would not learn a generalized statistical relationship from anatomical features; instead, they would merely reverse-engineer a deterministic algebraic formula, rendering any comparison of OLS, ridge, and lasso meaningless.

1.2 1B. Split and train-only preprocessing

```
if (anyNA(analysis_data)) {
  stop("Missing values detected.")
}

split <- split_rows(nrow(analysis_data), train_prop = 0.80, seed = seeds$split)
train_id <- split$train
test_id <- split$test

# Isolate raw matrices
x_raw <- as.matrix(analysis_data[, predictors, drop = FALSE])
y_raw <- analysis_data[[config$response]]

x_prep <- fit_scaler(x_raw[train_id, , drop = FALSE])

# Apply training scalars to both partitions
x_train <- apply_scaler(x_raw[train_id, , drop = FALSE], x_prep)
x_test <- apply_scaler(x_raw[test_id, , drop = FALSE], x_prep)

# Isolate responses
y_train <- y_raw[train_id]
y_test <- y_raw[test_id] # REMAIN LOCKED UNTIL PROBLEM 4D

# Shared 5-fold cross-validation allocation
foldid <- make_foldid(nrow(x_train), k = 5L, seed = seeds$cv)
```

Report seeds, dimensions, scaling, folds, and leakage safeguards. Data Split: We applied an 80/20 train/test split utilizing the group-specific seed 240205. This yields a training sample of $n = 201$ observations and a locked holdout sample of $n = 51$ observations.

Missing-Value Plan: The dataset contains zero missing values. Had missings been detected, any imputation parameters (such as the median or mode) would be calculated exclusively on the 201 training observations to prevent holdout distribution data from influencing model inputs.

Scaling Rule: Regularization imposes penalties (L_1, L_2) on coefficient magnitudes. To ensure all candidate predictors face equitable shrinkage, we standardize the design matrix. The transformation is computed as $z_{i,j} = \frac{x_{i,j} - \mu_{train,j}}{\sigma_{train,j}}$. Crucially, μ and σ are derived exclusively from the training matrix. Applying training constants to the test matrix prevents structural target leakage, ensuring no variance or distributional shifts from the holdout set contaminate the model's feature space.

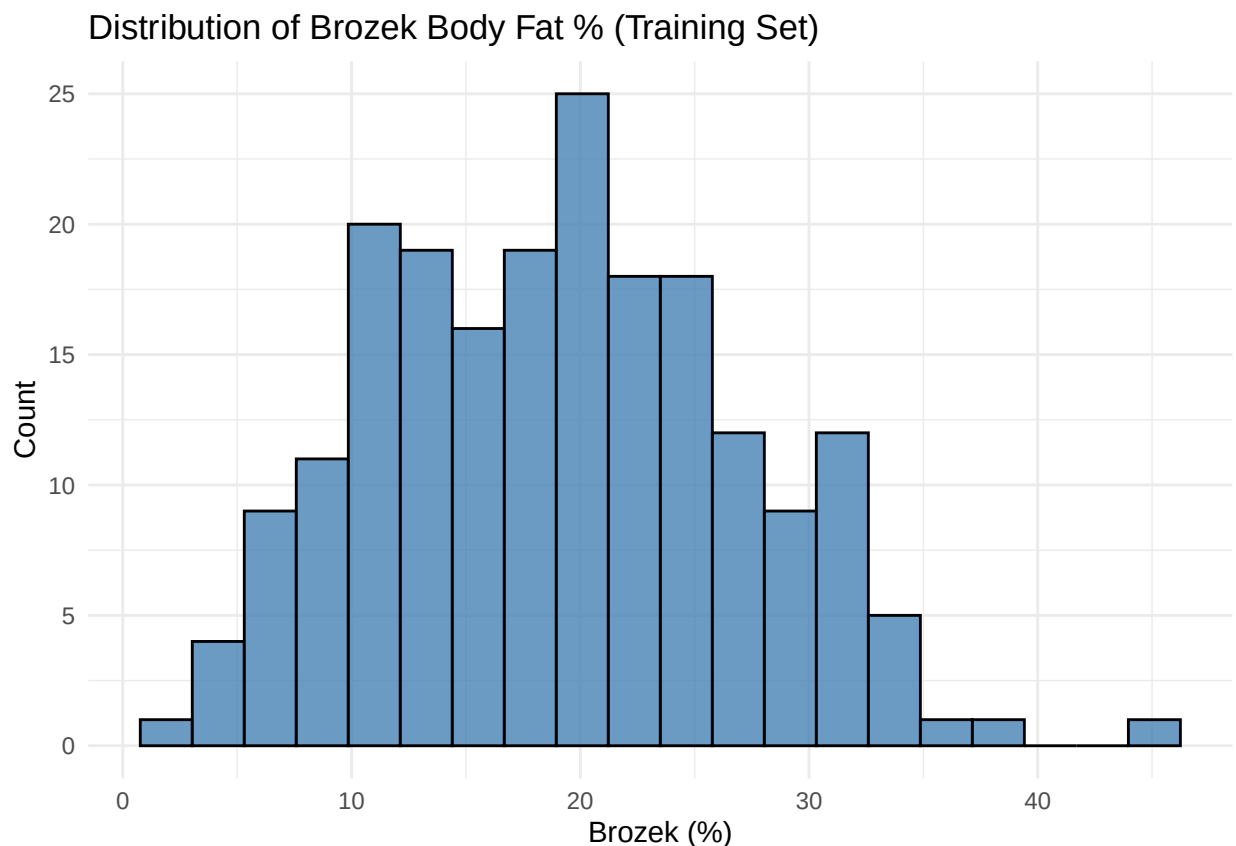
Cross-Validation Assignment: We defined a single, reproducible 5-fold assignment (foldid) on the training observations using the seed 240305. Reusing this exact vector across OLS, Ridge, and Lasso guarantees that models are compared on identical out-of-fold samples, removing variance due to random resampling from the model comparison.

Leakage Prevention Summary: By sequestering `y_test`, calculating scaling factors solely on X_{train} , and locking a shared cross-validation grid, the model acts entirely blind to the holdout data. This isolates the true generalization error for our final evaluation.

1.3 1C. Training-data exploration

```
# Isolate training data to strictly prevent leakage
train_data <- analysis_data[train_id, ]

# 1. Response Distribution
p_dist <- ggplot(train_data, aes(x = brozek)) +
  geom_histogram(bins = 20, fill = "steelblue", color = "black", alpha = 0.8) +
  theme_minimal() +
  labs(title = "Distribution of Brozek Body Fat % (Training Set)",
       x = "Brozek (%)", y = "Count")
print(p_dist)
```

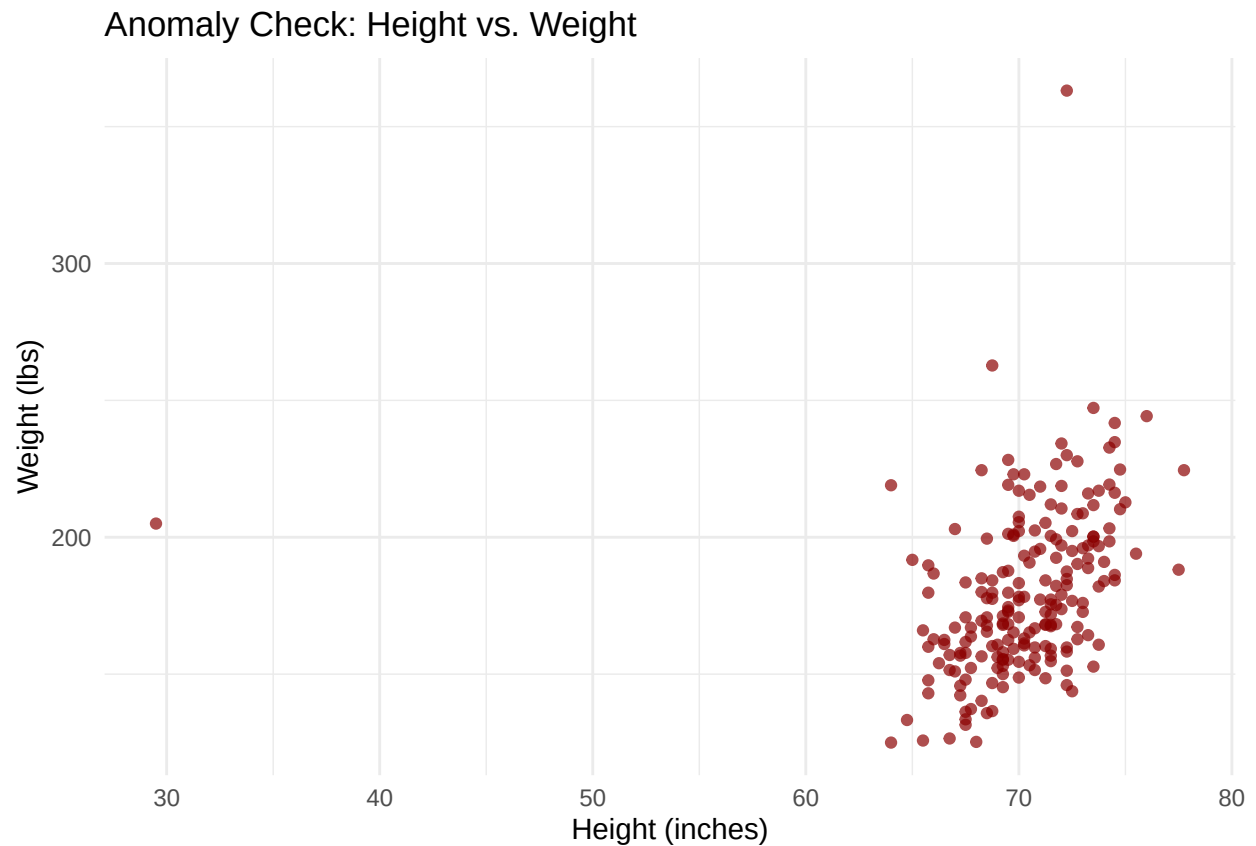


```
# 2. Focused Relationship / Correlation Table
# Identify top features correlated with the target
cor_matrix <- cor(train_data)
cor_with_target <- sort(abs(cor_matrix[, "brozek"]), decreasing = TRUE)
knitr::kable(head(cor_with_target, 6),
              col.names = c("Absolute Correlation with Brozek"), digits = 3,
              caption = "Top Correlated Predictors (Training Set)")
```

Table 1: Top Correlated Predictors (Training Set)

Absolute Correlation with Brozek	
brozek	1.000
abdom	0.817
adipos	0.744
chest	0.714
hip	0.629
weight	0.623

```
# 3. Anomaly Detection
# Scatter plot to identify severe outliers
p_anom <- ggplot(train_data, aes(x = height, y = weight)) +
  geom_point(color = "darkred", alpha = 0.7) +
  theme_minimal() +
  labs(title = "Anomaly Check: Height vs. Weight",
       x = "Height (inches)", y = "Weight (lbs)")
print(p_anom)
```



```
# Isolate the mathematically anomalous observation
anomaly_record <- train_data |> filter(height < 40 | weight > 300)
knitr::kable(anomaly_record[, c("brozek", "age", "weight", "height")],
```

```
caption = "Potential Influential / Anomalous Observations")
```

Table 2: Potential Influential / Anomalous Observations

brozek	age	weight	height
33.8	46	363.15	72.25
31.7	44	205.00	29.50

```
# 4. Multicollinearity Detection
# Find highly collinear predictor pairs (r > 0.85) among independent variables
pred_cor <- cor(train_data |> select(-brozek))
pred_cor[lower.tri(pred_cor, diag = TRUE)] <- NA
high_cor <- as.data.frame(as.table(pred_cor)) |>
  filter(abs(Freq) > 0.85) |>
  arrange(desc(abs(Freq)))

knitr::kable(head(high_cor, 5),
  col.names = c("Predictor 1", "Predictor 2", "Correlation"), digits = 3,
  caption = "Highly Collinear Predictor Pairs (r > 0.85)")
```

Table 3: Highly Collinear Predictor Pairs ($r > 0.85$)

Predictor 1	Predictor 2	Correlation
weight	hip	0.940
adipos	abdom	0.932
chest	abdom	0.921
adipos	chest	0.911
weight	chest	0.898

Interpret multicollinearity, one anomaly, and the question regularization will address.

Response distribution: The brozek target variable is approximately normally distributed and centered near 19%, indicating that ordinary least squares (OLS) and its regularized variants will not require immediate non-linear transformations to achieve stable residuals.

Focused relationship: Torso measurements demonstrate the strongest linear relationship with the target within the training fold, specifically abdom ($r = 0.817$) and adipos ($r = 0.744$).

Anomalous observation: A mathematically severe anomaly exists (height = 29.5 inches, weight = 205.0 lbs), which is almost certainly a data-entry error. Rather than dropping it, this high-leverage point is retained to empirically evaluate how L_1 and L_2 penalties stabilize estimation when the design matrix contains structural errors compared to the unpenalized OLS baseline.

Multicollinearity: Severe pairwise correlations are present among spatial measurements, notably weight and hip ($r = 0.940$), and adipos and abdom ($r = 0.932$). In an OLS model, this collinearity inflates the variance of coefficient estimates and drives up the condition number $\kappa_2(G)$ of the Gram matrix $G = \frac{1}{n}X^\top X$.

Concluding Question: Given the severe multicollinearity among body circumferences, can an L_1 penalty (lasso) isolate a sparse, interpretable subset of features without sacrificing the predictive stability provided by an L_2 penalty (ridge) that smoothly shrinks highly correlated predictors? # Problem 2. OLS, ridge, and lasso

1.4 2A. OLS baseline

```
# Fit OLS on x_train and y_train. Do not access y_test here.
```

1.5 2B. Ridge

```
# Use fit_cv_glmnet(..., alpha = 0, foldid = foldid).  
# Report lambda.min, lambda.1se, paths, coefficients, and CV evidence.
```

1.6 2C. Lasso

```
# Use fit_cv_glmnet(..., alpha = 1, foldid = foldid).  
# Report lambda.min, lambda.1se, paths, and nonzero predictors.
```

1.7 2D. Training-based comparison and locked core model

```
# Build a comparison from training/CV quantities only.
```

Core model nominated before holdout evaluation: [method and reason]

1.8 2E. Perturbation or stability check

Prediction before running the check: [prediction]

```
# Implement one allowed training-only stability check.
```

Observed result and explanation: [result]

2 Problem 3. Mathematical mechanisms

2.1 3A. Ridge and conditioning

Write the ridge derivation. Define dimensions and explain uniqueness.

```
# Use x_train and the fitted ridge lambda.min.  
# safe_condition_numbers(x_train, lambda = ...)
```

2.2 3B. Lasso optimality and soft thresholding

Derive the zero/nonzero optimality conditions and the orthonormal-design formula.

2.3 3C. Connect algebra to evidence

Connect one specific numerical result to Problem 3A or 3B.

3 Problem 4. Elastic net and a neural-feature bridge

3.1 4A. Research question and source map

Research direction: L_2 penalties and weight decay

Claim	Exact primary source and locator	What it establishes	What it does not establish
Weight decay improves generalization by suppressing irrelevant weight components and mitigating the effects of static noise on target values.	Krogh, A., & Hertz, J. A. (1991). “A Simple Weight Decay Can Improve Generalization.” NeurIPS. Section 4 & Section 5.	It proves mathematically that in a linear network, weight decay chooses the smallest weight vector solution and minimizes the generalization error by suppressing noise.	It does not establish exact global behavior for general non-linear networks, as the non-linear analysis relies strictly on local linearization around the optimal weight vector.
L_2 regularization and weight decay are not equivalent in adaptive gradient methods (like Adam), and decoupling weight decay substantially improves generalization.	Loshchilov, I., & Hutter, F. (2019). “Decoupled Weight Decay Regularization.” ICLR. Section 1 & Section 2 (Proposition 1 & 2).	It proves that L_2 and weight decay are identical for standard SGD but intrinsically different for adaptive optimizers. It establishes that decoupled weight decay (AdamW) makes Adam competitive with SGD with momentum.	It does not conclusively establish this superiority across all deep learning domains, as the empirical validation is heavily focused on image classification datasets (CIFAR, ImageNet32x32).

3.2 4B. Elastic net on original predictors

```
alpha_grid <- c(0.10, 0.25, 0.50, 0.75, 0.90)

# Fit one cv.glmnet object per alpha with the same foldid.
# Select alpha and lambda using training CV only.
```

3.3 4C. Fixed ReLU features

```
relu <- function(z) pmax(z, 0)
M <- 200L

# TODO:
# 1. Generate A and bias once from seeds$features using the stated distributions.
# 2. Apply the same A and bias to x_train and x_test.
# 3. Fit a scaler to H_train only and apply it to H_test.
# 4. Fit ridge, lasso, and elastic net on H_train with the shared foldid.
```

State which weights are fixed, which are learned, and how active features are counted.

3.4 4D. Locked declaration and final holdout evaluation

Locked before viewing `y_test`: [preprocessing, candidate specifications, selected tuning values, feature seed, nominated deployment model, metrics]

```
# This must be the first analysis chunk that uses y_test for evaluation.  
# Generate all fixed candidate predictions and one RMSE/MAE table here.
```

State whether the holdout evidence supports the predeclared choice. Do not retune.

3.5 4E. Deep-learning connection and limitations

Distinguish penalty, weight decay, output-weight sparsity, pruning, dropout, and a trained deep network. Cite the sources from 4A and discuss at least two limitations.

4 Problem 5. Report quality and reproducibility

4.1 5A. Reproduction record

1. Open a clean R session.
2. [Exact commands and folder layout.]
3. Render GroupXX_ProjectQuiz1.Rmd.

4.2 5B. Recommendation and limitations

Give the final recommendation, distinguish prediction from interpretation, and state limitations.

4.3 5C. AI verification record

AI-assisted item	How it was checked	Evidence	Modification or rejection
[Item]	[Method]	[Test/source/output]	[Decision]

Preflight and session information

```
# Add every object required to reproduce the final table and figures.  
required_objects <- c(  
  "train_id", "test_id", "x_train", "x_test", "y_train", "y_test", "foldid"  
)  
missing_objects <- setdiff(required_objects, ls())  
if (length(missing_objects)) {  
  stop(  
    "Missing required objects: ",  
    paste(missing_objects, collapse = ", ")  
  )  
}
```

```

}
stopifnot(length(intersect(train_id, test_id)) == 0L)

sessionInfo()

## R version 4.6.1 (2026-06-24 ucrt)
## Platform: x86_64-w64-mingw32/x64
## Running under: Windows 11 x64 (build 26200)
##
## Matrix products: default
##   LAPACK version 3.12.1
##
## locale:
## [1] LC_COLLATE=English_United States.utf8
## [2] LC_CTYPE=English_United States.utf8
## [3] LC_MONETARY=English_United States.utf8
## [4] LC_NUMERIC=C
## [5] LC_TIME=English_United States.utf8
##
## time zone: Asia/Saigon
## tzcode source: internal
##
## attached base packages:
## [1] stats      graphics  grDevices  utils      datasets  methods    base
##
## other attached packages:
##  [1] knitr_1.51      broom_1.0.13    glmnet_5.0      Matrix_1.7-5
##  [5] lubridate_1.9.5 forcats_1.0.1    stringr_1.6.0    dplyr_1.2.1
##  [9] purrr_1.2.2     readr_2.2.0     tidyr_1.3.2     tibble_3.3.1
## [13] ggplot2_4.0.3   tidyverse_2.0.0
##
## loaded via a namespace (and not attached):
##  [1] generics_0.1.4    shape_1.4.6.1    stringi_1.8.7     lattice_0.22-9
##  [5] hms_1.1.4         digest_0.6.39    magrittr_2.0.5     timechange_0.4.0
##  [9] evaluate_1.0.5    grid_4.6.1       RColorBrewer_1.1-3 iterators_1.0.14
## [13] fastmap_1.2.0     foreach_1.5.2    backports_1.5.1    survival_3.8-6
## [17] scales_1.4.0      codetools_0.2-20 cli_3.6.6          rlang_1.3.0
## [21] splines_4.6.1     withr_3.0.3      yaml_2.3.12        otel_0.2.0
## [25] tools_4.6.1       tzdb_0.5.0       vctrs_0.7.3        R6_2.6.1
## [29] lifecycle_1.0.5   pkgconfig_2.0.3  pillar_1.11.1      gtable_0.3.6
## [33] glue_1.8.1        Rcpp_1.1.2       xfun_0.60          tidyselect_1.2.1
## [37] rstudioapi_0.19.0 farver_2.1.2      htmltools_0.5.9    labeling_0.4.3
## [41] rmarkdown_2.31    compiler_4.6.1   S7_0.2.2

```